

Lab 11: Connect the Commerce Web App

Maps to: A3, A4, A6

Use fetch to power a responsive dashboard for products, customers and orders through FastAPI.

AI Vibe Coding Assets

- ai_vibe.py - OpenAI Python SDK runner
- mock-data.json - synthetic request and test context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - first-run and refinement prompts
- working-files/generated/ - isolated AI output for review

First-Run Prompt

Generate an accessible responsive HTML/CSS/JavaScript Northstar Commerce dashboard that uses fetch with the supplied FastAPI contract. Implement product/customer creation, order placement, KPI refresh and inventory rendering without page reload; keep live secrets out of committed code; render structured errors and empty/loading/failure states.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
pytest -q
```

Detailed procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 7: Run the completed API and open the root demo page.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 8: Inspect loadProducts() and trace GET /api/v1/products in the Network panel.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 9: Wire product and customer forms to POST JSON and render the returned records.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 10: Place an order and confirm the dashboard updates order count, revenue and stock.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 11: Display structured API errors in the accessible status region.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 12: Test desktop and narrow-screen layouts plus empty, loading, validation and server-error states.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Acceptance criteria

The browser reads/writes only through FastAPI, SQLite state changes correctly and the UI remains usable at 375px and desktop widths.